DSC 257R: Unsupervised learning

Homework 9

1. *Representations based on clustering.* Suppose that we have a data set $X = \{x_1, \ldots, x_n\}$ consisting of points in a metric space $(\mathcal{X}, d)$. We cluster the data to get k cluster centers $\mu_1, \ldots, \mu_k \in \mathcal{X}$. Here are a couple of the ways in which we can create a new representation of the data based on this clustering:

   (a) Represent each point by its cluster label, i.e., $\phi(x) = \arg\min_j d(x, \mu_j)$.

   (b) Represent each point by its distance to each of the centers, i.e., $\phi(x)_j = d(x, \mu_j)$.

   In each of these cases, precisely specify the space in which the representation $\phi(x)$ lies (e.g. $\mathbb{R}^n$).

   Note: When we have a hierarchical clustering, we can get more elaborate multi-scale representations.

2. Each of the following matrices gives distances (not squared distances) between a set of three points. In each case, say whether the distances can be realized in Euclidean space. If they can, show how by specifying three vectors with exactly the given interpoint distances. If they can't, give a reason.

$$\begin{bmatrix} 0 & 1 & 2 \\ 1 & 0 & 1 \\ 2 & 1 & 0 \end{bmatrix}, \quad \begin{bmatrix} 0 & 1 & 3 \\ 1 & 0 & 1 \\ 3 & 1 & 0 \end{bmatrix}.$$

3. Consider the following three points in $\mathbb{R}^3$:

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \; y = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}, \; z = \begin{bmatrix} 0 \\ -2 \\ -4 \end{bmatrix}$$

   (a) Let D be the matrix of squared interpoint distances. What is D?

   (b) What is the appropriate H matrix for this situation?

   (c) Compute $-\frac{1}{2}HDH$. What do you get?

   (d) Is the answer in (c) the correct Gram matrix for the data?

4. A set of n points in $\mathbb{R}^d$ is orthonormal: that is, each point has unit length and the points are orthogonal to one another.

   (a) What is the Gram matrix for this data? Be sure to specify its dimension clearly.

   (b) What is the matrix of squared interpoint distances?

5. *Experiments with multidimensional scaling.* The file `cities.txt` contains distances (as the crow flies) between ten European cities. We will use multidimensional scaling to obtain a two-dimensional embedding of these cities.

   (a) Show the 2-d embedding of the distance matrix obtained by cMDS. Here are the steps to follow:

- Write a function that implements the classical MDS algorithm from the lecture slides, taking as input a matrix of interpoint distances D and a target embedding dimension k. In order to perform the eigendecomposition, you can use `np.linalg.eigh`, although note that this returns eigenvalues in *increasing* rather than decreasing order.
- Write a function that takes a 2-d embedding of the points and plots them, annotating each point with the corresponding city name. The function `matplotlib.pyplot.annotate` is useful for these annotations.

(b) The embedding returned by cMDS (or any other multidimensional scaling method) is not unique: any rotation or translation of the points will produce the same interpoint distances. For this reason, your plot in (a) is unlikely to correspond to our own map of Europe. Show a rotated version of points that fits our own map more closely. Here are the steps to follow:

- Write a function that takes a 2-d data set and an angle, and rotates each of the points through that angle. To do this, recall that a 2-d point $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ can be rotated counterclockwise θ radians by multiplying it by the rotation matrix

$$\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}.$$

- Play around with the angle until you find something suitable.

(c) There are many other algorithms for multidimensional scaling. Some of them work by defining a "stress function" that captures the disparity between the specified interpoint distances and a given embedding; this stress function is then minimized using gradient descent. One such approach is implemented in `sklearn.manifold.MDS`. Use this to produce an embedding of the distances (use the setting `dissimilarity='precomputed'`), and once again find a suitable rotation. Plot the embedded points, annotated with city names, before and after rotation.